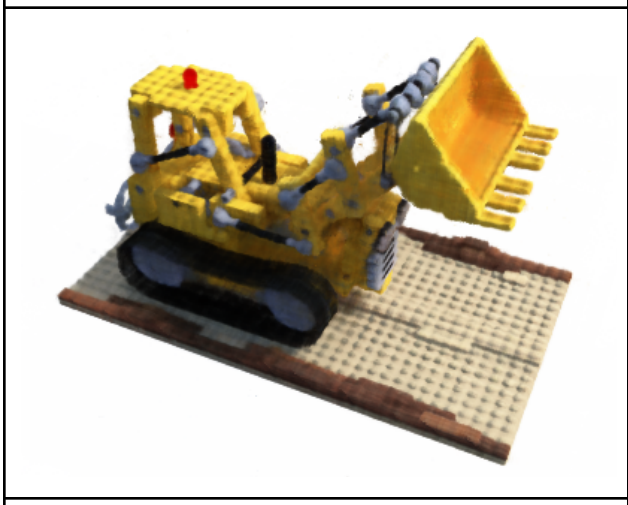
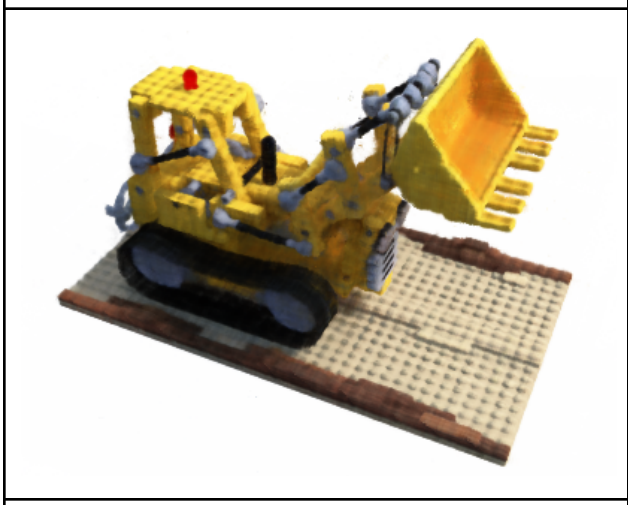
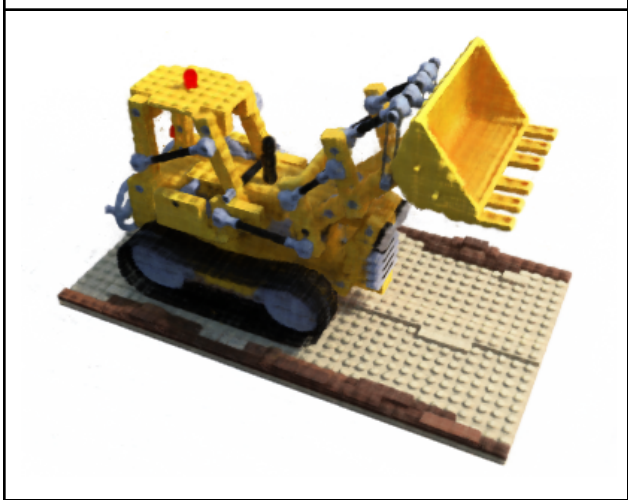
1. Briefly describe your implementation
    a. Positional Encoding (encoder.py):
        i. Implemented sinusoidal embedding with log-linear frequency sampling and fixed a closure bug in the lambda functions to avoid late binding issues.
    b. NeRF MLP (nerf.py):
        i. Built the 8-layer MLP with a skip connection at layer 4 that concatenates the original positional encoding back into the network, plus separate alpha and RGB branches where RGB additionally conditions on view directions.
    c. Volume Rendering (renderer.py):
        i. Implemented raw2alpha transmittance formula and quadrature weights using cumprod with a leading 1 and 1e-10 epsilon for numerical stability, then integrated RGB, depth, disparity and accumulation maps from those weights.
    d. Key debugging challenge:
        i. Disparity map produced NaN values for background rays (where acc_map ~ 0 causes 0/0) leading to black disp videos via np.max returning NaN. This was inherited from the original NeRF codebase and was resolved with np.nan_to_num before normalization.

2. Include some of the representative renderings for different iterations in training. Briefly comment on how the rendering quality is improving during training.
    a. Lego (25K iterations):
        i. PSNR rises rapidly from 4.5 dB to ~18 dB by iter 500, then steadily climbs to ~26-28 dB by iter 25K with diminishing returns as the network shifts from learning coarse geometry to refining high-frequency details.
        Per-batch PSNR shows high variance throughout (e.g. swinging ±3 dB between adjacent logged iterations), since each batch covers only a small random subset of rays rather than a full image. This is reflected in the representative renderings since the largest difference in image fidelity of the following images is between 4000 and 12000 iterations.
    b. Fern (50K iterations):
        i. My runtime disconnected around 40K so resuming from a 40K checkpoint at ~25.3 dB, the network gains only ~1 dB over the additional 10K iterations, indicating it had largely converged. The real-world LLFF scene presents additional challenges over synthetic data such as pose estimation noise, no clean background, and view-dependent lighting which cap the achievable PSNR lower than lego despite more total training iterations.

4000 iterations	10000 iterations
	
12000 iterations	20000 iterations
	
16000 iterations	30000 iterations

20000 iterations	40000 iterations
	
25000 iterations	50000 iterations